

Fiche 308 — Contrôle du flux : `if`, `switch`, `for`, `while`, `repeat`

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 9 « Grouping, loops and conditional execution » (§9.1 expressions groupées, §9.2.1 <code>if</code> , §9.2.2 <code>for</code> , <code>repeat</code> , <code>while</code>)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, §3.2 « Control structures » — §3.2.1 <code>if</code> , §3.2.2 boucles, §3.2.3 <code>repeat</code> , §3.2.4 <code>while</code> , §3.2.5 <code>for</code> , §3.2.6 <code>switch</code> ; §10.3.5 « Separators » (la règle du <code>else</code>)
Difficulté	Intermédiaire — court chapitre, mais R n'y ressemble pas aux autres langages
Temps d'étude estimé	1 h
Prérequis	Fiches 300 à 306
Concepts clés	langage d'expressions, valeur d'un bloc , <code>if</code> / <code>else</code> comme expression , <code>&&</code> et <code> </code> en court-circuit , <code>ifelse()</code> vectorisé, la règle du <code>else</code> en fin de ligne , <code>for</code> et sa variable survivante , <code>while</code> , <code>repeat</code> et <code>break</code> , <code>next</code> , valeur NULL invisible d'une boucle, <code>switch()</code> numérique et caractère, <code>split()</code> , l'avertissement sur les boucles
À retenir en priorité	<code>if</code> est une expression dont on peut prendre la valeur · <code>&</code> / <code> </code> vectorisés , <code>&&</code> / <code> </code> non · <code>ifelse()</code> pour le cas vectoriel · <code>} else {</code> sur la même ligne · « <i>code that takes a "whole object" view is likely to be both clearer and faster</i> ».

Vue d'ensemble

R EST UN LANGAGE D'EXPRESSIONS

« its ONLY command type is a function or expression WHICH RETURNS A RESULT »

meme une ASSIGNATION est une expression -- d'ou `x <- y <- 0`

UN BLOC `{ e1; ... ; em }` sa valeur est celle de la DERNIERE expression
 c'est une expression -> il s'imbrique partout

CONDITIONNELLE `if (cond) e2 else e3` -- cond doit valoir UNE SEULE valeur

	<code>y <- if (cond) a else b</code> -- on prend sa VALEUR sans else et cond fausse -> NULL
VECTORISEE	<code>ifelse(condition, a, b)</code> -- longueur de CONDITION a et b RECYCLES au besoin
COURT-CIRCUIT	<code>& </code> element par element, VECTORISES <code>&& </code> vecteurs de LONGUEUR UN, 2e argument evalue SI NECESSAIRE
BOUCLES	<code>for (nom in vecteur) expr</code> nom SURVIT a la boucle <code>while (cond) expr</code> <code>repeat expr</code> -- SEULE sortie : <code>break</code> <code>break</code> quitte la boucle la plus interne <code>next</code> passe a l'iteration suivante VALEUR d'une boucle : NULL, rendu INVISIBLEMENT
AIGUILLAGE	<code>switch(valeur, ...)</code> numerique : la n-ieme branche, sinon NULL caractere : le nom EXACT, sinon le default
L'AVERTISSEMENT	« <code>for()</code> loops are used in R code MUCH LESS OFTEN than in compiled languages. Code that takes a "WHOLE OBJECT" view is likely to be both CLEARER AND FASTER in R. »

Le problème posé. « *R is an expression language in the sense that its only command type is a function or expression which returns a result. Even an assignment is an expression whose result is the value assigned, and it may be used wherever any expression may be used ; in particular multiple assignments are possible.* » (§9.1)

INTUITION — POURQUOI CETTE PHRASE CHANGE TOUT.

Dans la plupart des langages, une *instruction* fait quelque chose et une *expression* vaut quelque chose ; ce sont deux catégories. **En R il n'y en a qu'une.** `if` vaut quelque chose. Un bloc `{ }` vaut quelque chose. Une assignation vaut quelque chose. C'est pourquoi on peut écrire `y <- if (cond) log(1+x) else log(x)` — construction impossible dans un langage à instructions.

● Concept 1 — Le bloc, et la valeur d'un groupe

DÉFINITION (§9.1).

« **Commands may be grouped together in braces, `{expr_1; ...; expr_m}`, in which case the value of the group is the result of the last expression in the group evaluated. Since such a group is also an expression it may, for example, be itself included in parentheses and used as part of an even larger expression, and so on.** »

COMPLÉMENT (R LANGUAGE DEFINITION §3.2).

« *Statements can be grouped together using braces ...* **A group of statements is sometimes called a block. Single statements are evaluated when a new line is typed at the end of the syntactically complete statement. Blocks are not evaluated until a new line is entered after the closing brace.** »

```
{ x <- 0
  x + 5 }
# [1] 5
```

Deux conséquences pratiques. (1) La **dernière expression** d'une fonction en est la valeur de retour — c'est ce qui rend `return()` facultatif (fiche 309). (2) À la console, R **attend l'accolade fermante** avant d'évaluer quoi que ce soit, ce qui explique la suite de `+` du prompt de continuation.

« *Both semicolons and new lines can be used to separate statements. A semicolon always indicates the end of a statement while a new line may indicate the end of a statement. If the current statement is not syntactically complete new lines are simply ignored by the evaluator.* » (§3.2)

● Concept 2 — `if` : une conditionnelle qui vaut quelque chose

DÉFINITION (§9.2.1).

« *The language has available a conditional construction of the form `if (expr_1) expr_2 else expr_3`, where `expr_1` must evaluate to a single logical value and the result of the entire expression is then evident.* »

COMPLÉMENT — LES RÈGLES EXACTES (R LANGUAGE DEFINITION §3.2.1).

« First, `statement1` is evaluated to yield `value1`. If `value1` is a logical vector with first element `TRUE` then `statement2` is evaluated. If the first element ... is `FALSE` then `statement3` is evaluated. If `value1` is a numeric vector then `statement3` is evaluated when the first element ... is zero and otherwise `statement2` is evaluated. Only the first element of `value1` is used. All other elements are ignored. If `value1` has any type other than a logical or a numeric vector an error is signalled. »

Valeur de la condition	Effet
logique, 1 ^{er} élément <code>TRUE</code>	branche <code>then</code>
logique, 1 ^{er} élément <code>FALSE</code>	branche <code>else</code>
numérique, 1 ^{er} élément zéro	branche <code>else</code>
numérique, 1 ^{er} élément non nul	branche <code>then</code>
autre type	erreur

« Only the first element ... is used. All other elements are ignored » décrit une sémantique historique. Les versions récentes de R traitent une condition de longueur > 1 comme **une erreur** ; c'est bien ce que la règle voulait prévenir. Dans les deux cas, **la leçon est la même** : la condition d'un `if` doit être **une seule valeur**. Pour un test élément par élément, c'est `ifelse()` (concept 4).

2.1 `if` est une expression — la conséquence utile

MÉTHODE (§3.2.1).

« `if / else` statements can be used to avoid numeric problems such as taking the logarithm of a negative number. Because `if / else` statements are the same as other statements you can assign the value of them. The two examples below are equivalent » :

```
if (any(x <= 0)) y <- log(1+x) else y <- log(x)
y <- if (any(x <= 0)) log(1+x) else log(x)
```

La seconde forme est meilleure : `y` n'est assigné **qu'une fois**, à un seul endroit ; il n'y a pas de branche où l'on aurait pu oublier l'assignation.

« **The `else` clause is optional.** The statement `if(any(x <= 0)) x <- x[x <= 0]` is valid. » Et si la condition est fausse sans `else`, « **no statement will be evaluated and `NULL` is returned** » — l'expression a quand même une valeur.

2.2 La règle du `else`, et pourquoi elle existe

Règle (§3.2.1). « **When the `if` statement is not in a block the `else`, if present, must appear on the same line as the end of `statement2`. Otherwise the new line at the end of `statement2` completes the `if` and yields a syntactically complete statement that is evaluated. A simple solution is to use a compound statement wrapped in braces, putting the `else` on the same line as the closing brace.** »

C'est la règle déjà croisée en fiche 300, du côté du parseur (§10.3.5) : « **Special rules apply to the `else` keyword : inside a compound expression, a newline before `else` is discarded, whereas at the outermost level, the newline terminates the `if` construction and a subsequent `else` causes a syntax error.** »

POURQUOI CETTE BIZARRERIE ?

Le manuel le dit : « **This somewhat anomalous behaviour occurs because `R` should be usable in interactive mode and then it must decide whether the input expression is complete, incomplete, or invalid as soon as the user presses `RET`.** »

À la console, `R` **ne peut pas attendre** de voir s'il y aura un `else` : il doit répondre immédiatement. Dans un bloc, il sait qu'il y a une suite, donc il peut patienter. **La règle est le prix de l'interactivité.**

La forme sûre, valable partout :

```
if (cond) {  
  ...  
} else if (autre) {  
  ...  
} else {
```

```
...  
}
```

L'enchaînement (§3.2.1). « `if / else` statements *can be nested ... One of the even numbered statements will be evaluated ... The odd numbered statements are evaluated, in order, until one evaluates to* `TRUE` ... *There is no limit to the number of* `else if` *clauses that are permitted.* » Et si tout est faux sans `else` final, `NULL` est rendu.

● Concept 3 — `&&` et `||` : le court-circuit

DÉFINITION (§9.2.1).

« The "**short-circuit**" operators `&&` and `||` are often used as part of the condition in an `if` statement. Whereas `&` and `|` apply element-wise to vectors, `&&` and `||` apply to vectors of length one, and only evaluate their second argument if necessary. »

	<code>&</code> et <code> </code>	<code>&&</code> et <code> </code>
Portée	élément par élément	une seule valeur
Longueur du résultat	celle du plus long (recyclage)	1
Second argument	toujours évalué	« <i>only ... if necessary</i> »
Usage	filtrer un vecteur	condition d'un <code>if</code>

MÉTHODE — POURQUOI LE COURT-CIRCUIT EST INDISPENSABLE.

Le second opérande de `&&` **n'est pas évalué** quand le premier suffit à décider. Cela permet d'écrire une condition dont **la seconde moitié serait invalide** si la première était fausse :

```
if (length(x) > 0 && x[1] > 0) ...
```

Avec `&`, `x[1]` serait évalué **même sur un vecteur vide**. Avec `&&`, il ne l'est que si `length(x) > 0`. **L'ordre des tests devient porteur de sens** — le garde-fou vient toujours en premier.

Rappel de la table de la fiche 301 (*R Language Definition* §3.1.4) : `&` est « *And, binary, **vectorized*** », `&&` « *And, binary, **not vectorized*** ».

● Concept 4 — `ifelse()` : la conditionnelle vectorisée

DÉFINITION (§9.2.1).

*« There is a **vectorized version of the** `if / else` **construct, the** `ifelse` **function**. This has the form `ifelse(condition, a, b)` and returns a **vector of the same length as** `condition`, with elements `a[i]` **if** `condition[i]` **is true, otherwise** `b[i]` (where `a` and `b` are recycled as necessary). »*

Trois différences avec `if`, toutes importantes :

	<code>if (cond) a else b</code>	<code>ifelse(cond, a, b)</code>
<code>cond</code>	une seule valeur	un vecteur
Longueur du résultat	celle de la branche choisie	celle de <code>cond</code>
Évaluation des branches	une seule	les deux — puis on choisit

La troisième différence est un piège de performance et de correction. `ifelse()` évalue `a` et `b` en entier, sur tout le vecteur, avant de choisir. Un `ifelse(x > 0, log(x), 0)` calcule donc `log(x)` y compris sur les valeurs négatives, avec les avertissements `NaN` qui vont avec. La sélection ne protège pas du calcul.

Rappel — l'exemple de l'annexe A (fiche 300). `w <- ifelse(Mod(w) > 1, 1/w, w)` : la longueur du résultat est celle de `Mod(w) > 1`, donc **100** ; et `1/w` est calculé partout, y compris là où on garde `w`. C'est sans conséquence ici, puisque `1/w` est toujours défini.

● Concept 5 — Les trois boucles

DÉFINITION (§9.2.2).

« There is also a **for loop construction** which has the form `for (name in expr_1) expr_2`, where **name** is the loop variable. **expr_1** is a **vector expression** (often a sequence like `1:20`), and **expr_2** is often a **grouped expression** with its sub-expressions written in terms of the dummy **name**. **expr_2** is repeatedly evaluated as **name** ranges through the values in the vector result of **expr_1**. »

COMPLÉMENT (R LANGUAGE DEFINITION §3.2.5).

« **vector** can be either a vector or a list. For each element in **vector** the variable **name** is set to the value of that element ... **A side effect is that the variable name still exists after the loop has concluded and it has the value of the last element of vector that the loop was evaluated for.** »

La variable de boucle survit. Après `for (i in 1:10) ...`, **i** vaut **10** et existe toujours dans l'espace de travail. Ce n'est pas une portée de bloc comme dans d'autres langages — c'est une variable ordinaire (fiche 310). D'où le désordre de l'espace de travail dont parle le §6.3.3 (fiche 306).

Les deux autres formes (§9.2.2) :

```
repeat expr
while (condition) expr
```

Règle (§3.2.3). « The **repeat** statement causes **repeated evaluation of the body until a break is specifically requested**. This means that **you need to be careful when using repeat because of the danger of an infinite loop**. ... **When using repeat, statement must be a block statement** : you need to both **perform some computation and test whether or not to break**, and usually this requires two statements. »

Règle (§3.2.4). « The `while` statement is very similar to the `repeat` statement ... `statement1` is evaluated and **if its value is `TRUE` then `statement2` is evaluated. This process continues until `statement1` evaluates to `FALSE` .** »

5.1 `break` et `next`

DÉFINITION (§9.2.2).

« The `break` statement can be used to terminate any loop, possibly abnormally. This is the only way to terminate `repeat` loops. »

« The `next` statement can be used to discontinue one particular cycle and skip to the "next". »

Précision (§3.2.2). « The `break` statement causes an exit from the innermost loop that is currently being executed. The `next` statement immediately causes control to return to the start of the loop ... No statement below `next` in the current loop is evaluated. »

`break` ne quitte que la boucle **la plus interne**. Dans deux boucles imbriquées, il ne sort que de celle du dedans. R n'a pas d'étiquette de boucle : sortir des deux demande un drapeau, ou une réécriture.

5.2 La valeur d'une boucle

Règle (§3.2.2). « The value returned by a loop statement is always `NULL` and is returned invisibly. »

C'est la seule construction du langage qui **ne rend rien d'utile**. On ne peut donc pas écrire `x <- for (...)` : une boucle n'agit que par **effet de bord**. C'est cohérent avec l'avertissement du concept 6 — une boucle est, en R, un aveu qu'on n'a pas trouvé l'opération vectorisée.

5.3 L'exemple du cours, et `split()`

Exemple (§9.2.2). *« suppose `ind` is a vector of class indicators and we wish to **produce separate plots of `y` versus `x` within classes**. One possibility here is to use `coplot()`, which will produce an array of plots corresponding to each level of the factor. **Another way to do this, now putting all plots on the one display**, is as follows »* :

```
xc <- split(x, ind)
yc <- split(y, ind)
for (i in 1:length(yc)) {
  plot(xc[[i]], yc[[i]])
  abline(lsfrit(xc[[i]], yc[[i]]))
}
```

« (Note the function `split()` which produces a list of vectors obtained by splitting a larger vector according to the classes specified by a factor. This is a useful function, mostly used in connection with boxplots.) »

CONTRÔLE — LIRE LE CODE.

`split()` rend **une liste** (fiche 306), d'où les `[[ ]]` : `xc[[i]]` est **le vecteur** de la classe *i*, tandis que `xc[i]` serait une **sous-liste** inutilisable par `plot`. C'est l'illustration exacte du « *It is very important to distinguish* » du §6.1.

Et `1:length(yc)` est ici sans danger, `yc` ayant au moins un élément — mais la fiche 301 rappelle que `seq(along = yc)` est la forme sûre.

(Note 1 du §9.2.2 : `coplot()` est « *to be discussed later* », ou l'on peut utiliser `xyplot` du paquet `Lattice`.)

● Concept 6 — L'avertissement du cours sur les boucles

Warning (§9.2.2, encadré dans le cours). « `for()` loops are used in R code much less often than in compiled languages. Code that takes a "whole object" view is likely to be both clearer and faster in R. »

C'est le fil conducteur de tout ce qui précède, énoncé pour la première fois de façon explicite :

Au lieu de boucler	Écrire
sur les éléments d'un vecteur	une opération vectorisée (fiche 301)
pour appliquer une fonction par groupe	<code>tapply()</code> (fiche 304)
pour évaluer une fonction sur toutes les paires	<code>outer()</code> (fiche 305)
pour remplir des cases dispersées	une matrice d'index (fiche 305)
pour compter des croisements	<code>table()</code> (fiche 305)

COMPLÉMENT (R LANGUAGE DEFINITION §3.2.2).

« *R provides other functions for implicit looping such as `tapply`, `apply`, and `lapply`. In addition many operations, especially arithmetic ones, are vectorized so you may not need to use a loop.* »

Le cours en a donné une démonstration chiffrée au §5.5 (fiche 305) : sur le problème des déterminants, « The "obvious" way of doing this problem with `for` loops ... is **so inefficient as to be impractical** ».

« **Whole object view** » est l'expression à retenir : penser à l'objet entier, pas à ses éléments un par un.

● Concept 7 — switch()

DÉFINITION (R LANGUAGE DEFINITION §3.2.6).

« *Technically speaking, `switch` is just another function, but its semantics are close to those of control structures of other programming languages.* » La syntaxe est `switch(statement, list)`, « *where the elements of `list` may be named* ».

7.1 Le cas numérique

Règle (§3.2.6). « *First, `statement` is evaluated and the result, `value`, obtained. If `value` is a number between 1 and the length of `list` then the corresponding element of `list` is evaluated and the result returned. If `value` is too large or too small `NULL` is returned.* »

```
x <- 3
switch(x, 2+2, mean(1:10), rnorm(5))
# [1] 2.2903605 2.3271663 -0.7060073 1.3622045 -0.2892720

switch(2, 2+2, mean(1:10), rnorm(5))
# [1] 5.5

switch(6, 2+2, mean(1:10), rnorm(5))
# NULL
```

Hors bornes donne `NULL`, pas une erreur. Comme pour `x$inexistent` (fiche 302), le silence est le comportement.

7.2 Le cas caractère

Règle (§3.2.6). « *If `value` is a character vector then the element of ... with a name that exactly matches `value` is evaluated. If there is no match a single unnamed argument will be used as a default. If no default is specified, `NULL` is returned.* »

```

y <- "fruit"
switch(y, fruit = "banana", vegetable = "broccoli", "Neither")
# [1] "banana"

y <- "meat"
switch(y, fruit = "banana", vegetable = "broccoli", "Neither")
# [1] "Neither"

```

« **exactly matches** » — pas d'appariement partiel ici, contrairement à `$` et `[[` (fiche 302). Et **le défaut est l'argument sans nom** : c'est sa position syntaxique, non un mot-clé, qui le désigne.

L'usage courant (§3.2.6). « *A common use of `switch` is to branch according to the character value of one of the arguments to a function.* »

```

centre <- function(x, type) {
  switch(type,
    mean    = mean(x),
    median  = median(x),
    trimmed = mean(x, trim = .1))
}
x <- rcauchy(10)
centre(x, "mean")      # 0.8760325
centre(x, "median")

```

Remarquer qu'il n'y a pas de défaut ici : `centre(x, "autre")` rend `NULL`, silencieusement. C'est un défaut de robustesse que la fiche 309 apprendra à combler (`match.arg` , `stop`).

Énoncé. Le code suivant calcule, pour chaque état, le revenu moyen des comptables du chapitre 4. Le réécrire dans l'esprit de l'avertissement du §9.2.2.

```

res <- numeric(0)
for (s in levels(statef)) {

```

```
res[s] <- mean(incomes[statef == s])
}
```

Étape 1 — *ce que fait la boucle*. Elle parcourt les **huit niveaux**, filtre `incomes` par une condition logique (fiche 302), calcule la moyenne, et **allonge `res` en écrivant hors des bornes** (fiche 303, §3.2).

Étape 2 — *pourquoi c'est correct mais mauvais*. Le résultat est juste. Mais `res` **est réalloué à chaque tour** — l'écriture hors bornes recrée l'objet (fiche 302, mécanisme ``*tmp*``) —, la variable de boucle `s` **survit** à la boucle, et l'intention (« une moyenne par groupe ») est noyée dans la mécanique.

Étape 3 — *reconnaître le patron*. « Appliquer une fonction à chaque groupe défini par un facteur » **est** la définition de `tapply()` (fiche 304).

Étape 4 — *la réécriture*.

```
res <- tapply(incomes, statef, mean)
```

Étape 5 — *vérifier l'équivalence*. `tapply` rend « *a structure of **the same length as the** `Levels attribute`* », **étiquetée par les niveaux** — exactement ce que la boucle construisait à la main par `res[s] <- ...`. Les huit valeurs sont celles vérifiées en fiche 304 : act 44,500 · nsw 57,333 · ... · wa 52,250.

Étape 6 — *ce qu'on a gagné*. Une ligne au lieu de quatre ; **pas de variable résiduelle** ; l'allocation faite **une seule fois** ; et surtout, le code **dit ce qu'il fait**. C'est le « *clearer and faster* » de l'avertissement, dans cet ordre — **la clarté d'abord**.

Étape 7 — *quand la boucle reste légitime*. L'exemple du §9.2.2 lui-même en est un : la boucle sur `split()` **trace des graphiques**, c'est-à-dire qu'elle n'agit que par **effet de bord**. Rien à accumuler, rien à vectoriser. C'est précisément le cas où « *the value returned by a loop statement is always NULL* » n'est pas une gêne.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« selon que la condition est vraie ou fausse »	<code>if / else</code> — condition d'une seule valeur
« affecter une valeur ou une autre »	<code>y <- if (cond) a else b</code> — <code>if</code> est une expression
« élément par élément »	<code>ifelse(cond, a, b)</code>
« erreur : la condition a une longueur > 1 »	il fallait <code>ifelse()</code> , ou <code>any()</code> / <code>all()</code>
« ne pas évaluer le second test si le premier échoue »	<code>&&</code> / <code> </code>
« erreur d'indice sur un vecteur vide dans un <code>if</code> »	mettre le garde-fou en premier avec <code>&&</code>
« erreur de syntaxe sur <code>else</code> »	<code>} else {</code> sur la même ligne
« répéter un nombre connu de fois »	<code>for</code>
« répéter tant que »	<code>while</code>
« répéter jusqu'à ce que »	<code>repeat</code> + <code>break</code>
« sortir de la boucle »	<code>break</code> — seulement la plus interne
« passer au tour suivant »	<code>next</code>
« ma variable de boucle traîne encore »	elle survit — c'est normal
« <code>x <- for(...)</code> ne marche pas »	une boucle rend <code>NULL</code> , invisiblement
« aiguiller selon une chaîne »	<code>switch()</code> — appariement exact
« mon <code>switch</code> rend <code>NULL</code> »	valeur hors bornes , ou aucun défaut
« découper un vecteur selon un facteur »	<code>split()</code> — rend une liste
« ma boucle est lente »	« code that takes a whole object view ... »

Comment résoudre ce type d'exercice

Protocole « écrire une conditionnelle correcte » — 4 étapes.

1. **Quelle est la longueur de la condition ?** Une valeur → `if` ; un vecteur → `ifelse()` .
2. Si l'on veut **une valeur**, écrire `y <- if (...) ... else ...` plutôt que d'assigner dans chaque branche.
3. Utiliser `&&` / `||` dans un `if` , `&` / `|` pour filtrer — et **placer le garde-fou en premier**.
4. Écrire `} else {` sur la même ligne, toujours.

Protocole « faut-il vraiment une boucle ? » — 4 étapes.

1. **Y a-t-il une opération vectorisée ?** Arithmétique, comparaison, `paste` ... → fiche 301.
2. **Est-ce un traitement par groupe ?** → `tapply()` , `split()` + `sapply` .
3. **Est-ce une évaluation sur toutes les paires ?** → `outer()` .
4. Si la réponse est non trois fois — ou si la boucle n'agit que par **effet de bord** (tracer, écrire) — alors la boucle est légitime.

Protocole « boucle sûre » — 4 étapes.

1. Parcourir avec `seq(along = v)` , jamais `1:length(v)` (fiche 301).
2. **Pré-allouer** le résultat plutôt que de l'agrandir à chaque tour.
3. `repeat` **exige** un `break` — le vérifier avant d'exécuter.
4. Nettoyer la **variable de boucle** si elle gêne : elle **survit**.

● Common mistakes

Erreur	Correction
Passer un vecteur en condition de <code>if</code>	<code>if</code> veut une seule valeur → <code>ifelse()</code> , <code>any()</code> , <code>all()</code>
Utiliser <code>&</code> dans la condition d'un <code>if</code>	<code>&&</code> — non vectorisé et court-circuit
Placer le garde-fou en second dans un <code>&&</code>	l'ordre porte le sens : le test protecteur d'abord
Croire que <code>ifelse()</code> n'évalue qu'une branche	il évalue les deux , puis choisit
Attendre d' <code>ifelse()</code> la longueur de <code>a</code>	c'est celle de condition
Mettre <code>else</code> en début de ligne au niveau externe	erreur de syntaxe → <code>} else {</code>
Croire cette règle arbitraire	elle vient de l'interactivité du parseur
Croire qu'un <code>if</code> sans <code>else</code> ne vaut rien	il rend <code>NULL</code>
Croire que la variable de boucle disparaît	elle survit , avec la dernière valeur
Croire que <code>break</code> sort de toutes les boucles	seulement de la plus interne
Écrire un <code>repeat</code> sans <code>break</code>	boucle infinie — c'est la seule sortie
Utiliser <code>repeat</code> avec une seule instruction	« <code>statement</code> must be a block statement »
Affecter le résultat d'une boucle	elle rend <code>NULL</code> , invisiblement
Faire grossir un vecteur à chaque tour	pré-allouer , ou vectoriser
Attendre un appariement partiel de <code>switch()</code>	il apparie exactement
Oublier le défaut d'un <code>switch</code> caractère	sans lui, <code>NULL</code> silencieux
Croire qu'un <code>switch</code> numérique hors bornes lève une erreur	il rend <code>NULL</code>
Utiliser <code>xc[i]</code> au lieu de <code>xc[[i]]</code> après <code>split()</code>	<code>split</code> rend une liste
Boucler par réflexe	« <i>much less often than in compiled languages</i> »

Ultimate Review

Le langage. « *R is an expression language in the sense that its only command type is a function or expression which returns a result. Even an assignment is an expression whose result is the value assigned* » — d'où les assignations multiples.

Le bloc. {expr_1; ...; expr_m} — « *the value of the group is the result of the last expression in the group evaluated* » · un bloc **est une expression**, donc s'imbrique · à la console, « **blocks are not evaluated until a new line is entered after the closing brace** ».

if . « **expr_1 must evaluate to a single logical value** » · logique → premier élément ; **numérique** → **else** **si zéro** ; **autre type** → **erreur** · **else** **facultatif** ; sans lui et condition fausse → **NULL** · **on peut en prendre la valeur** : `y <- if (any(x <= 0)) log(1+x) else log(x)` · enchaînement **else if** **sans limite**, les impaires évaluées **dans l'ordre jusqu'au premier TRUE** .

La règle du **else** . « *When the if statement is not in a block the else , if present, must appear on the same line as the end of statement2 .* » Raison : « *R should be usable in interactive mode and then it must decide ... as soon as the user presses RET* ». Solution : `} else { .`

Court-circuit. « *Whereas & and | apply element-wise to vectors, && and || apply to vectors of length one, and only evaluate their second argument if necessary.* » D'où le garde-fou : `if (length(x) > 0 && x[1] > 0) .`

ifelse() . « *a vectorized version of the if / else construct* » · rend « *a vector of the same length as condition* » · `a[i]` si vrai, sinon `b[i]` , « *where a and b are recycled as necessary* » · les deux branches sont évaluées.

Boucles. `for (name in expr_1) expr_2` — `expr_1` peut être **un vecteur ou une liste** ; « *the variable name still exists after the loop has concluded and it has the value of the last element* » · `while (cond) expr` · `repeat expr` — « *until a break is specifically requested* », `statement` **doit être un bloc**, danger de boucle infinie · `break` sort de « *the innermost loop* » et est « *the only way to terminate repeat loops* » · `next` « *immediately causes control to return to the start* » ; « *no statement below next ... is evaluated* » · « *The value returned by a loop statement is always NULL and is returned invisibly.* »

L'avertissement. « `for()` loops are used in R code much less often than in compiled languages. Code that takes a "whole object" view is likely to be both clearer and faster in R. »

Les substituts : **vectorisation**, `tapply`, `apply`, `lapply`, `outer`, matrice d'index, `table`.

`split()` . « produces a list of vectors obtained by splitting a larger vector according to the classes specified by a factor » — d'où l'accès par `[[ ]]` .

`switch()` . « just another function, but its semantics are close to those of control structures » · **numérique** : la n -ième branche si $1 \leq n \leq$ longueur, sinon `NULL` · **caractère** : le nom qui apparie exactement, sinon l'unique argument sans nom sert de défaut, sinon `NULL` · usage courant : « branch according to the character value of one of the arguments to a function ».

Active Recall

« R is an expression language in the sense that its only command type is a function or expression which returns a result. Even an assignment is an expression whose result is the value assigned, and it may be used wherever any expression may be used ; in particular multiple assignments are possible. » (§9.1)

Il n'y a pas deux catégories — instructions et expressions — comme dans la plupart des langages. Tout vaut quelque chose.

La conséquence pratique la plus utile est donnée au §3.2.1 : « Because `if / else` statements are the same as other statements you can assign the value of them » :

```
y <- if (any(x <= 0)) log(1+x) else log(x)
```

`y` est assigné **une seule fois**, à un seul endroit — impossible d'oublier une branche.

« the value of the group is the result of the last expression in the group evaluated » (§9.1). Et « since such a group is also an expression it may ... be used as part of an even larger expression ».

À la console : « *Single statements are evaluated when a new line is typed at the end of the syntactically complete statement. **Blocks are not evaluated until a new line is entered after the closing brace.*** » (§3.2)

```
{ x <- 0
  x + 5 }
# [1] 5
```

Deux conséquences : la dernière expression d'une fonction en est la valeur de retour (fiche 309) ; et le prompt `+` persiste tant que l'accolade n'est pas fermée.

« `expr_1` **must evaluate to a single logical value** » (§9.2.1). Le détail (§3.2.1) :

Condition	Effet
logique, 1 ^{er} élément <code>TRUE</code>	branche <code>then</code>
logique, 1 ^{er} élément <code>FALSE</code>	branche <code>else</code>
numérique, 1 ^{er} élément zéro	branche <code>else</code>
numérique, non nul	branche <code>then</code>
autre type	erreur

« ***Only the first element ... is used. All other elements are ignored.*** » décrit une sémantique historique ; les versions récentes de R **lèvent une erreur** sur une condition de longueur > 1 — ce que la règle cherchait précisément à éviter. **Dans les deux cas, la condition doit valoir une seule valeur.**

Sans `else` et condition fausse : « ***no statement will be evaluated and `NULL` is returned*** ».

« ***When the `if` statement is not in a block the `else`, if present, must appear on the same line as the end of `statement2`. Otherwise the new line at the end of `statement2`***

completes the `if` and yields a syntactically complete statement that is evaluated. » (§3.2.1)

La raison est l'interactivité (§10.3.5) : « *This somewhat anomalous behaviour occurs because **R should be usable in interactive mode** and then **it must decide whether the input expression is complete, incomplete, or invalid as soon as the user presses RET.*** »

Au niveau externe, R **ne peut pas attendre** de voir si un `else` arrive. Dans un bloc, il sait qu'il y a une suite.

La solution : « *use a compound statement wrapped in braces, **putting the `else` on the same line as the closing brace*** » — `} else {` .

« *Whereas `&` and `|` apply element-wise to vectors, `&&` and `||` apply to vectors of length one, and only evaluate their second argument if necessary.* » (§9.2.1)

Le cas décisif :

```
if (length(x) > 0 && x[1] > 0) ...
```

Avec `&&` , si `length(x) > 0` est faux, `x[1]` **n'est jamais évalué**. Avec `&` , il le serait — sur un vecteur vide, produisant `NA` (fiche 302 : un indice positif hors bornes rend `NA`), et le `if` échouerait.

L'ordre des tests devient porteur de sens : le garde-fou vient **toujours** en premier. C'est ce qu'on appelle le court-circuit.

*« **a vectorized version of the `if / else` construct** ... returns **a vector of the same length as `condition`** , with elements `a[i]` if `condition[i]` is true, otherwise `b[i]` (where `a` and `b` are recycled as necessary). »* (§9.2.1)

	<code>if</code>	<code>ifelse()</code>
Condition	une valeur	un vecteur
Longueur du résultat	celle de la branche	celle de <code>cond</code>
Branches évaluées	une seule	les deux

Le piège est la troisième ligne. `ifelse(x > 0, log(x), 0)` calcule `log(x)` **sur tout le vecteur**, y compris les négatifs — avec les `NaN` et les avertissements. **La sélection ne protège pas du calcul.**

« **A side effect is that the variable `name` still exists after the loop has concluded and it has the value of the last element of `vector` that the loop was evaluated for.** » (*R Language Definition §3.2.5*)

Après `for (i in 1:10) ...`, `i` **vaut 10** et reste dans l'espace de travail. R n'a **pas** de portée de bloc : la variable de boucle est une variable ordinaire (fiche 310).

C'est l'une des sources du désordre que la règle 4 du §6.3.3 demande de nettoyer (fiche 306) : « *remove all unwanted variables from the workspace* ».

À noter aussi : « **`vector` can be either a vector or a list** ».

Forme	Sortie
<code>for (name in vecteur) expr</code>	naturelle, à la fin du vecteur — ou <code>break</code>
<code>while (cond) expr</code>	quand <code>cond</code> devient <code>FALSE</code> — ou <code>break</code>
<code>repeat expr</code>	<code>break</code> uniquement

« **The `break` statement can be used to terminate any loop, possibly abnormally. This is the only way to terminate `repeat` loops.** » (§9.2.2) Et « **causes an exit from the innermost loop** » (§3.2.2).

« **The `next` statement can be used to discontinue one particular cycle and skip to the "next".** » ; « **No statement below `next` in the current loop is evaluated.** »

`repeat` : « *you need to be careful ... because of the danger of an infinite loop* »,
et « *statement must be a block statement* ».

« *The value returned by a loop statement is always `NULL` and is returned invisibly.* » (*R Language Definition §3.2.2*)

C'est la seule construction du langage qui ne rend rien d'utile — alors même que « *R is an expression language in the sense that its only command type is a function or expression which returns a result* ».

La cohérence est là : une boucle n'agit que par **effet de bord**. Et c'est exactement le sens de l'avertissement du §9.2.2 — en R, écrire une boucle est le plus souvent l'aveu qu'on n'a pas trouvé l'opération « objet entier » qui aurait rendu un résultat.

« *`for()` loops are used in R code much less often than in compiled languages. Code that takes a "whole object" view is likely to be both clearer and faster in R.* »

L'expression à retenir est « **whole object view** » — penser à l'objet entier, pas à ses éléments.

Les substituts, cités par la *R Language Definition* (§3.2.2) : *« R provides other functions for **implicit looping** such as `tapply`, `apply`, and `lapply`. In addition **many operations, especially arithmetic ones, are vectorized so you may not need to use a loop.** »*

Et la démonstration chiffrée est au §5.5 (fiche 305) : sur le problème des déterminants, la version à boucles est « *so inefficient as to be impractical* ».

Noter l'ordre des adjectifs : « *clearer and faster* » — la clarté d'abord.

« *`split()` ... produces a list of vectors obtained by splitting a larger vector according to the classes specified by a factor. This is a useful function, mostly used in connection with boxplots.* » (§9.2.2)

Le résultat étant **une liste**, l'accès à un élément se fait par `[[ ]]` (fiche 306) :

```
xc <- split(x, ind)
for (i in 1:length(yc)) {
  plot(xc[[i]], yc[[i]])
  abline(lsfite(xc[[i]], yc[[i]]))
}
```

`xc[i]` rendrait **une sous-liste** d'un élément — que `plot()` ne peut pas traiter comme un vecteur de coordonnées. C'est l'illustration du « *It is very important to distinguish `Lst[[1]]` from `Lst[1]`* » du §6.1.

Et cette boucle-ci **est légitime** : elle ne fait que **tracer**, donc n'agit que par effet de bord.

« *Technically speaking, `switch` is just another function* » (§3.2.6).

Numérique : « *If `value` is a number between 1 and the length of `List` then the corresponding element ... is evaluated ... If `value` is too large or too small `NULL` is returned.* »

```
switch(2, 2+2, mean(1:10), rnorm(5)) # 5.5
switch(6, 2+2, mean(1:10), rnorm(5)) # NULL
```

Caractère : « *the element of ... with a name that exactly matches `value` is evaluated. If there is no match a single unnamed argument will be used as a default. If no default is specified, `NULL` is returned.* »

```
switch("meat", fruit = "banana", vegetable = "broccoli", "Neither") # "Neither"
```

« **exactly matches** » — pas d'appariement partiel, contrairement à `$` et `[[`. Et le défaut est l'argument sans nom, désigné par sa forme, pas par un mot-clé.

« A common use of `switch` is to branch according to the character value of one of the arguments to a function. »

Flashcards

Question	Réponse
Qu'est-ce qu'un « langage d'expressions » ?	Son seul type de commande rend un résultat
Une assignation est-elle une expression ?	Oui — sa valeur est la valeur assignée
Que permet cela ?	Les assignations multiples
Quelle est la valeur d'un bloc ?	Celle de la dernière expression évaluée
Un bloc est-il une expression ?	Oui — il s'imbrique
Quand un bloc est-il évalué à la console ?	Après la ligne suivant l'accolade fermante
Que doit valoir la condition d'un <code>if</code> ?	Une seule valeur logique
Condition numérique nulle ?	Branche <code>else</code>
Condition d'un autre type ?	Erreur
<code>else</code> est-il obligatoire ?	Non
Valeur d'un <code>if</code> sans <code>else</code> , condition fausse ?	<code>NULL</code>
Peut-on assigner la valeur d'un <code>if</code> ?	Oui
L'écriture recommandée ?	<code>y <- if (cond) a else b</code>
Combien de clauses <code>else if</code> au maximum ?	Aucune limite
Dans quel ordre sont-elles évaluées ?	Dans l'ordre , jusqu'au premier <code>TRUE</code>
Où doit se trouver <code>else</code> hors bloc ?	Sur la même ligne que la fin de <code>statement2</code>
Pourquoi cette règle ?	R doit décider dès la frappe de RET
La forme sûre ?	<code>} else {</code>
<code>&</code> contre <code>&&</code> ?	Vectorisé contre une seule valeur
Que fait le court-circuit ?	Le 2 ^e argument n'est évalué que si nécessaire
Où placer le garde-fou dans un <code>&&</code> ?	En premier

Question	Réponse
La version vectorisée de <code>if</code> ?	<code>ifelse(cond, a, b)</code>
Longueur de son résultat ?	Celle de <code>condition</code>
<code>a</code> et <code>b</code> sont-ils recyclés ?	Oui , au besoin
Combien de branches <code>ifelse()</code> évalue-t-il ?	Les deux
Les trois boucles de R ?	<code>for</code> , <code>while</code> , <code>repeat</code>
Syntaxe du <code>for</code> ?	<code>for (name in vecteur) expr</code>
Sur quoi peut-on boucler ?	Un vecteur ou une liste
Que devient la variable de boucle ?	Elle survit , avec la dernière valeur
Comment sortir d'un <code>repeat</code> ?	<code>break</code> uniquement
<code>repeat</code> accepte-t-il une instruction simple ?	Non — un bloc
De quelle boucle <code>break</code> sort-il ?	La plus interne
Que fait <code>next</code> ?	Il passe au tour suivant
Ce qui suit <code>next</code> dans le tour ?	N'est pas évalué
Que rend une boucle ?	<code>NULL</code> , invisiblement
Que dit l'avertissement du §9.2.2 ?	Les boucles sont bien moins fréquentes qu'ailleurs
Quelle vision recommande-t-il ?	La vision « objet entier »
Quels en sont les bénéfices ?	« <i>clearer and faster</i> »
Trois fonctions de boucle implicite ?	<code>tapply</code> , <code>apply</code> , <code>lapply</code>
Que fait <code>split()</code> ?	Découpe un vecteur en liste , selon un facteur
Son usage le plus courant ?	Les boîtes à moustaches
Pourquoi <code>xc[[i]]</code> et non <code>xc[i]</code> ?	<code>split</code> rend une liste
Qu'est-ce que <code>switch</code> , techniquement ?	Une fonction comme une autre

Question	Réponse
<code>switch</code> numérique hors bornes ?	<code>NULL</code>
<code>switch</code> caractère : quel appariement ?	Exact — pas de partiel
Comment désigne-t-on le défaut ?	C'est l' unique argument sans nom
Sans défaut et sans correspondance ?	<code>NULL</code>
L'usage le plus courant de <code>switch</code> ?	Aiguiller selon un argument caractère d'une fonction
Autre fonction pour tracer par classe ?	<code>coplot()</code> , ou <code>xyplot</code> du paquet <code>lattice</code>